```
assert = require 'assert'

{addWord, removeWord} = require './word_utils'

fact = (description, fn) ->
  try
    fn()
    console.log "#{description}: OK"
  catch e
    console.error "#{description}: \n#{e.stack}"
    throw e

fact "addWord adds a word", ->
  input = "product special"
  expectedOutput = "product special popular"
  actualOutput = addWord input, "popular"

  assert.equal expectedOutput, actualOutput


fact "removeWord removes a word and surrounding whitespace", ->
  tests = [
    initial: "product special"
    replace: "special"
    expected: "product"
  ,
    initial: "product special"
    replace: "spec"
    expected: "product special"
  ]

  for {initial, replace, expected} in tests
    assert.equal removeWord(initial, replace), expected
```

A destructuring require from the two functions exported from word_utils.

The fact function takes two arguments: a string description of the test and the function that's to be tested. It wraps the test function call inside a try...catch, passing the test if no exception occurs and failing the test if an exception does occur.

Running this file outputs the description of each fact along with the result:

```
> coffee word_utils.spec.coffee
addWord adds a word: OK
removeWord removes a word and surrounding whitespace: OK
```

The `addWord` and `removeWord` functions are general solutions to part of a more specific problem of adding a class to a `<div>`. What would have happened had you dove straight in and written an `addWord` implementation like this?

```
addClass = (selector, newClass) ->
  element = document.querySelector selector
  if element.className?
    element.className = "#{element.className} #{newClass}"
```